

Property-based testing

In chapter 7 we worked through the design of a functional library for expressing parallel computations. There we introduced the idea that an API should form an *algebra*—that is, a collection of data types, functions over these data types, and importantly, *laws* or *properties* that express relationships between these functions. We also hinted at the idea that it might be possible to somehow check these laws automatically.

This chapter will take us toward a simple but powerful library for *property-based testing*. The general idea of such a library is to decouple the specification of program behavior from the creation of test cases. The programmer focuses on specifying the behavior of programs and giving high-level constraints on the test cases; the framework then automatically generates test cases that satisfy these constraints, and runs tests to ensure that programs behave as specified.

Although a library for testing has a very different purpose than a library for parallel computations, we'll discover that these libraries have a lot of surprisingly similar combinators. This similarity is something we'll return to in part 3.

8.1 A brief tour of property-based testing

As an example, in ScalaCheck (<http://mng.bz/n2j9>), a property-based testing library for Scala, a property looks something like this.

Listing 8.1 ScalaCheck properties

A generator of lists of integers between 0 and 100.

```
val intList = Gen listOf(Gen.choose(0,100))
val prop =
  forAll(intList)(ns => ns.reverse.reverse == ns) &&
```

A property that specifies the behavior of the `List.reverse` method.

Check that reversing a list twice gives back the original list.

A property which is obviously false.

```
forall(intList)(ns => ns.headOption == ns.reverse.lastOption)
val failingProp = forall(intList)(ns => ns.reverse == ns)
```

Check that the first element becomes the last element after reversal.

And we can check properties like so:

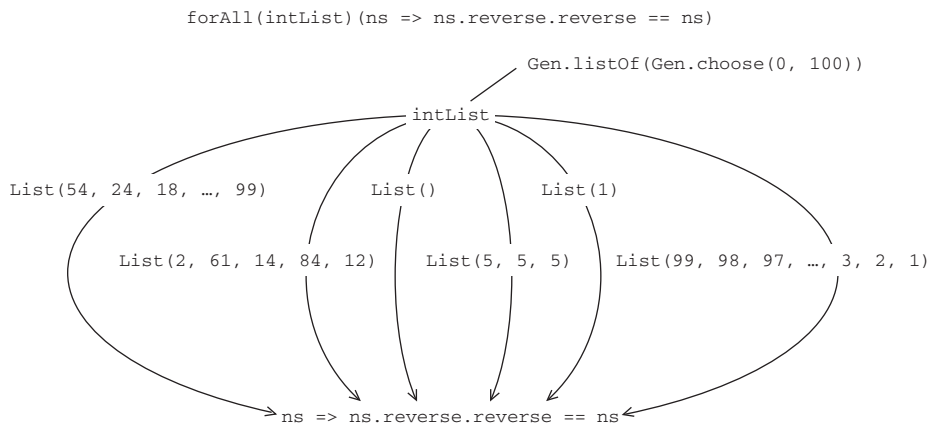
```
scala> prop.check
+ OK, passed 100 tests.

scala> failingProp.check
! Falsified after 6 passed tests.
> ARG_0: List(0, 1)
```

Here, `intList` is not a `List[Int]`, but a `Gen[List[Int]]`, which is something that knows how to generate test data of type `List[Int]`. We can *sample* from this generator, and it will produce lists of different lengths, filled with random numbers between 0 and 100. Generators in a property-based testing library have a rich API. We can combine and compose generators in different ways, reuse them, and so on.

The function `forall` creates a *property* by combining a generator of type `Gen[A]` with some predicate of type `A => Boolean`. The property asserts that all values produced by the generator should satisfy the predicate. Like generators, properties can also have a rich API. In this simple example we've used `&&` to combine two properties. The resulting property will hold only if neither property can be *falsified* by any of the generated test cases. Together, the two properties form a partial specification of the correct behavior of the `reverse` method.¹

Generators and properties



A `Gen` object generates a variety of different objects to pass to a Boolean expression, searching for one that will make it false.

¹ The goal of this sort of testing is not necessarily to fully specify program behavior, but to give greater confidence in the code. Like testing in general, we can always make our properties more *complete*, but we should do the usual cost-benefit analysis to determine if the additional work is worth doing.

When we invoke `prop.check`, `ScalaCheck` will randomly generate `List[Int]` values to try to find a case that falsifies the predicates that we've supplied. The output indicates that `ScalaCheck` has generated 100 test cases (of type `List[Int]`) and that they all satisfied the predicates. Properties can of course fail—the output of `failingProp.check` indicates that the predicate tested false for some input, which is helpfully printed out to facilitate further testing or debugging.

**EXERCISE 8.1**

To get used to thinking about testing in this way, come up with properties that specify the implementation of a `sum: List[Int] => Int` function. You don't have to write your properties down as executable `ScalaCheck` code—an informal description is fine. Here are some ideas to get you started:

- Reversing a list and summing it should give the same result as summing the original, nonreversed list.
- What should the sum be if all elements of the list are the same value?
- Can you think of other properties?

**EXERCISE 8.2**

What properties specify a function that finds the maximum of a `List[Int]`?

Property-based testing libraries often come equipped with other useful features. We'll talk more about some of these features later, but just to give an idea of what's possible:

- *Test case minimization*—In the event of a failing test, the framework tries smaller sizes until it finds the *smallest* test case that also fails, which is more illuminating for debugging purposes. For instance, if a property fails for a list of size 10, the framework tries smaller lists and reports the smallest list that fails the test.
- *Exhaustive test case generation*—We call the set of values that could be produced by some `Gen[A]` the *domain*.² When the domain is small enough (for instance, if it's all even integers less than 100), we may exhaustively test all its values, rather than generate sample values. If the property holds for all values in a domain, we have an actual *proof*, rather than just the absence of evidence to the contrary.

² This is the same usage of “domain” as the domain of a function (<http://mng.bz/ZP8q>)—generators describe possible inputs to functions we'd like to test. Note that we'll also still sometimes use “domain” in the more colloquial sense, to refer to a subject or area of interest, for example, “the domain of functional parallelism” or “the error-handling domain.”